Railway Scheduling Report

Jonathan Drathjer, Arne Frenzel, and Leander von Gottberg

Institut für Informatik, Universität Potsdam, Potsdam, Germany
drathjer@uni-potsdam.de, frenzel5@uni-potsdam.de, gottberg@uni-potsdam.de

Abstract. The Flatland problem was proposed to gain an understanding of machine learning in the context of train scheduling. For the last few years, a group at the University of Potsdam has tried to solve the problem using Answer Set Programming (ASP). This report will specifically look at how ASP can be used to solve the Flatland problem for trains with different speeds. Furthermore, we will give an outlook on what needs to be done to transform our implementation of speed into a fully-fledged Flatland solution.

1 Problem of Flatland

Flatland is a multi-agent reinforcement learning (MARL) environment designed to address railway traffic management and train scheduling problems. It provides a simulated environment where agents can learn and optimize train movements, reducing congestion, minimizing delays, and ensuring efficient use of railway infrastructure. Developed by the Swiss Federal Railways (SBB) in collaboration with AIcrowd, Flatland serves as a testbed for developing and benchmarking advanced train scheduling algorithms.

Flatland represents a railway network using a discrete 2D grid where tracks, switches, and junctions are modeled explicitly. Each train in the system is considered an agent that must navigate the network while avoiding conflicts with other trains. The primary challenge in Flatland is to coordinate multiple agents efficiently so they reach their respective destinations without collisions or unnecessary delays.

2 Pushing the boundary

A research group at the University of Potsdam has worked with ASP to solve Flatland instances. But there are still issues that have not been addressed or implemented. Among those problems are:

- In Flatland, it is possible that trains have malfunctions. Therefore, the scheduling has to be dynamically changed.
- Trains can have different constant speeds.

In given ASP solutions every train has a speed of 1. Because that is not realistic we decided to focus our work on implementing different speeds for trains.

The speed of a train is defined as follows: speed = time needed for moving a train from on cell to another. For example, if a train with speed 4 wants to move from cell(1,1) to cell(1,2), that will take 4 time steps to execute. In the following you will see how our ASP implementation works and how the speed comes into play.

3 Implementation and solving of the Problems

Our implementation is a transition based approach. It defines the possible connections between tracks and concludes the following actions of a train, based on the trains position and direction. It also considers the speed of a train when generating the actions. The details are described in the following sections.

3.1 Connections

Before talking about connections we need to introduce the coordinate system Flatland uses. The (0,0) cell is in the top left corner with the first axis (X-axis) extending downward and the second axis (Y-axis) extending to the right as shown in figure 1.

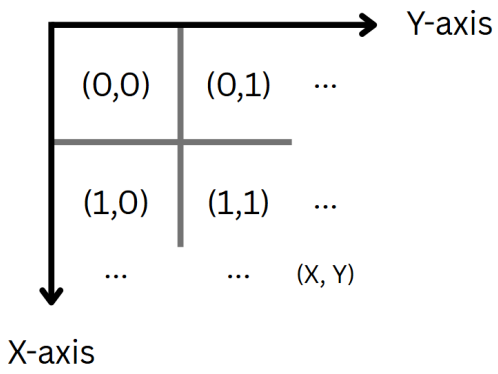

Fig. 1: Flatland coordinate system

The Flatland environment declares all possible tracks that a cell could be. It provides a bijective mapping between numbers and these track types. There are straights, curves and combinations or rotations of those track types. We structured the tracks hierarchically, where each complex track type would be a combination of basic track types. The basic track types are straights and curves of every rotation. We introduce a new predicate *connection((X, Y), DIR, STEP)* which gives each basic track type the direction a train needs to face to

use this *connection*. It also gives the STEP, which is just the coordinate change after using that *connection*. Both STEP and DIR will be explained in detail in the direction subsection 3.2. Each basic track gets two connection predicates, because a train could only enter the track from two directions. For example, a cell with a straight vertical track would get a connection derived from its cell:

```
1  connection((X, Y), n, (-1,0)) :- cell((X, Y), 32800).
2  connection((X, Y), s, (1, 0)) :- cell((X, Y), 32800).
```

Listing 1.1: Rules for connections

A train could enter the tracks facing north or facing south. The connections are now essential for determining in what direction the train can move. If the train is facing north, our *connection* predicate forces the train to continue moving north. If its facing south, the *connection* predicate also forces the train to continue moving south.

The complex track types are defined by combining the basic track types or by combining complex track types together. From a given complex track type A we derive two track types B, C that are lower in the hierarchy. The set of connections for A is the union of the connection sets for B and C. Based on this construction we get a tree with the basic track types at the leafs and the most complex track types at the root. An example of our structure is shown in Figure 2. Of course, you could use a different tree structure to reduce the depth of the tree, but the amount of leafs will remain the same. From the basic track types in the leafs we conclude the connections which the complex track type at the root of the tree has.

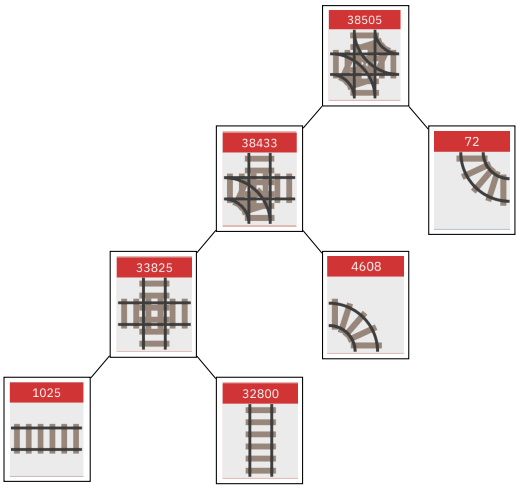

Fig. 2: Example tree structure of a complex track type 38505

3.2 Directions

As mentioned above, the *connection* predicate consists of a position, a direction, and a step. We will now explain what we mean by direction and step and how they are encoded.

The direction is a two-dimensional vector. It encodes the orientation that a train must have to be able to use the given connection. The mapping of $\{n, s, e, w\}$ to the vectors is given by the following table:

cardinal direction	n	s	w	e
vector representation	(-1, 0)	(1, 0)	(0, -1)	(0, 1)

Table 1: direction vector representation

This mapping follows an intuitive rule: If a train is oriented in the direction x and moves one step in that direction, the position after the move is given by the original position plus the vector representation of x. Let us consider an example from figure 3: A train is at position (1, 1) and faces n. It moves one step n so it is now at $(0, 1) = (1, 1) + (-1, 0)$ where (-1, 0) is the vector representation of n. With this we now say that each connection has a required incoming direction, encoded by this mapping. The reason why this certain representation is useful will become clear once we look at the action generation in subsection 3.6.

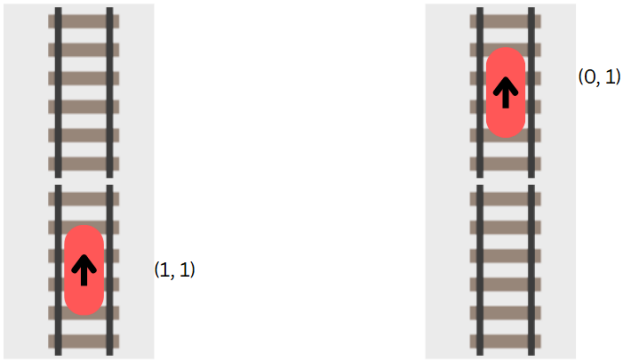

Fig. 3: direction example

The step also is a two-dimensional vector and follows a similar logic: If a train moves through a connection, the position after the move is given by the original position plus the step vector of that connection. Let us consider an example from figure 4: A train is at position (1, 1) facing n and there exists a connection from

(1, 1) to (1, 2) for trains facing n. Therefore, the step of this connection is (0, 1) since $(1,1) + (0,1) = (1,2)$. If we want to calculate the resulting position, we simply add the step (0, 1) to its position before the move. This will become useful when implementing how trains can move in subsection 3.4.

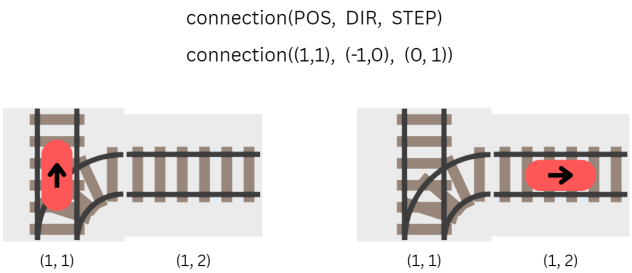

Fig. 4: step example

3.3 Starting Positions

From the given start predicate we derive our starting *thomases*. Thomas is our predicate for the moving trains. Why it makes sense to use it and not simply train will be mentionted in subsection 3.5. The spawn of a train at time step 0 is a bit of a special case in flatland. Because a train need to be ready to depart before it can move, a train which should move at T=0 really moves at T=1. Therefore every action of that train is delayed by one time step. To solve that problem we just let every train which would spawn at T=0 spawn at T=1.

3.4 Thomas Generation

The thomas generation of our implementation only consists of two rules:

```
1  {thomas(ID, (X+X',Y+Y'), T+V, (X', Y')) : connection((X, Y)
       , DIR, (X', Y'))} = 1 :- thomas(ID, (X, Y), T, DIR),
       end(ID, POS, DEADLINE), T+V<=DEADLINE, (X, Y) != POS,
       speed(ID, V).
2  :- not thomas(ID, POS, _, _), end(ID, POS, DEADLINE).
```

Listing 1.2: Rules for solving fucntion

Rule 2 gives the thomases a reason to move. Without it, they would have no reason to actually move to their destination. The rule says that for each *end* predicate there needs to exist the corresponding thomas on that position. Because we are only generating thomases up until their own deadline in Rule 1 this thomas implicitly reaches the destination before its own deadline. Rule 1

is the core of our speed implementation. The basic idea of this rule is that we generate the thomases inductively, meaning that from a thomas there follows the thomas that has moved on more time and so on. With a train speed of 1 you could say that from a thomas at time step T follows the thomas at time step T+1. If we remind ourselves that a thomas with speed V needs V time steps to move one cell, we can easily see how to implement the same idea as with a speed of 1. We simply state that from a thomas with speed V at time step T, there follows the thomas at time step T+V. With this intuition, let us take a look at all the possible ways a train could move. If the thomas is at position (X, Y) with orientation DIR we say that it must choose exactly one of the connections at (X, Y) for trains facing DIR. Because we implemented the STEP = (X', Y') in a way such that it gives the change in coordinates for using that connection, we can then say that thomas after using the connection is at (X+X', Y+Y'). It is important to note, that because trains can not move sideways or backwards, the STEP is equivalent to the direction of the thomas after it has moved. So the direction of the thomas is also (X', Y'). As mentioned above, we only generate thomases that are not over their own deadline. This not only makes grounding smaller but also makes it easier to reason with thomas predicates. To achieve that we simply check that T+V $\leq$ DEADLINE holds. Since Flatland automatically despawns trains as soon as they reach their destination we also forbid thomas generation after reaching the destination. For that we only generate a new thomas if the current position is not its destination.

3.5 Collision handling

The collision handling prevents trains from being scheduled in a way that leads to collisions. To exclude all solutions that contain collisions, our approach requires two rules.

```
1  :- train(ID, POS, T, DIR), train(ID', POS, T, DIR'), ID<ID
       '.
2  :- train(ID, POS, T, _), train(ID, POS', T+V, _), train(ID
       ', POS', T, _), train(ID', POS, T+V', _), ID<ID', speed
       (ID, V), speed(ID', V').
```

Listing 1.3: Rules for collision handling

The first rule forbids that two different trains have the same position at the same time step. This is not sufficient since trains are also not allowed to switch places. In this case the trains would collide in between two time steps which would not be avoided by the first rule. Rule 2 forbids that two trains at POS_1, POS_2 have the positions POS_2, POS_1 after both have moved once. This is necessary since this scenario can only occur if the trains went through each other.

A problem which comes with our speed implementation is that the thomases only exist when their speed allows them to make a move. In between the time steps, they do not exist, which makes them unsuited for collision handling. To keep the collision handling easy and efficient we decided to differentiate between

the predicate for the actions namely *thomas* and the *train* predicate for collision handling. The new predicate *train* can simply be concluded from the *thomas* predicate and can be filled up between the time steps. Now the collision handling can simply be done via *train* without interfering with *thomas*.

3.6 Action generation

```
1  action(train(ID), move_forward, T+1) :- POS!=POS', thomas(
       ID, POS', T+V, DIR'), thomas(ID, POS, T, DIR), #count{
       STEP : connection(POS, DIR, STEP)} = 1, speed(ID, V).
2  action(train(ID), move_forward, T+1) :- POS!=POS', thomas(ID
     , POS', T+V, DIR'), thomas(ID, POS, T, DIR), #count{ STEP
      : connection(POS, DIR, STEP)} >1, DIR' = DIR, speed(ID,
     V).
3  action(train(ID), move_left, T+1) :- POS!=POS', thomas(ID,
     POS', T+V, (X', Y')), thomas(ID, POS, T, (X, Y)), #count{
      STEP : connection(POS, (X, Y), STEP)} >1, (X * Y') - (X'
      * Y) > 0, speed(ID, V).
4  action(train(ID), move_right, T+1) :- POS!=POS', thomas(ID,
     POS', T+V, (X', Y')), thomas(ID, POS, T, (X, Y)), #count{
      STEP : connection(POS, (X, Y), STEP)} >1, (X * Y') - (X'
      * Y) < 0, speed(ID, V).
```

Listing 1.4: Clingo ASP code for the action-generation

In our implementation we used thomases to derive our movements. However the flatland environment needs actions for each train and not the positions. Therefore we need to extract the actions 'move forward', 'move left', 'move right' and 'wait' from our thomases. However the action cannot simply be concluded from the change in position and direction, since the action is also dependent on the track type. If a train changes the direction to the left, and the track type is a switch, the action would be a 'move left'. If a train however does the same move in a normal curve without a switch the action would be a normal move forward. In fact the change in direction of the train only matters on a switch, since otherwise there is only one way to go and the resulting action in the step should be 'move forward'. Therefore we can just count the number of possible connections our thomases can use. Rule 1 states, that if there is exactly one connection, we only can 'move forward'. Rule 2 says, that if the number of possible connections is greater than 1, the issue gets more complex. We now need to look at the change in direction of the thomas. If now the direction thomas faces does not change, the action is a 'move forward'. If the directions changes, the action is a 'move left' or a 'move right'. With our encoding of directions from subsection 3.2 it is possible to determine the correct action via the two-dimensional cross product. We need to find out whether the rotation that was done was clockwise ('move right') or counter clockwise ('move left'). Let us call the direction of thomas before the move DIR_1 and the one after the move DIR_2. If the cross product of DIR_1 and DIR_2 is greater than zero, the rotation was clockwise. If the cross product is

less than zero then the rotation was counterclockwise. Rule 3 and 4 implement this idea. If the cross product is zero, then the train is moving forward which is already handled in Rule 1 and 2.

3.7 Theory of Flatland relativity

agent	timestep	position	direction	status	given command
3	274	(30, 36)	n	moving	move forward
5	275	(33, 21)	w	moving	move forward
3	276	(30, 36)	n	moving	move forward
5	277	(33, 20)	w	moving	move forward
3	278	(30, 37)	e	moving	move forward
5	279	(33, 20)	w	moving	move forward
3	280	(30, 37)	e	moving	move forward

Table 2: Example of the problem. Exert from the paths.csv

When working with different speeds in Flatland we encountered some strange behavior for the time steps of slower trains. The problem occurs when there is no active train with speed equal to 1 in the whole environment. In this case there are time steps in which nothing happens. Flatland does not approve of those time steps and it gets rid of them by simply pulling the next action from a later time step. However the train is then not able to fulfill that action since its speed does not allow it to move yet. This results in the action getting wasted and trains not arriving in Flatland, although they receive the correct actions from our solver. An example for this issue can be seen in Table 3. In this exert from a paths.csv file there are just two trains left which have not arrived yet. These are train 3 and train 5 - both with a speed of 4 and we will focus on train 3. In time step 274 train 3 receives 'move forward'. In 275 train 5 receives an action as well. In the next 2 time steps nothing happens theoretically. Because of that flatland pulls forward the next action. This action is the 'move forward' that was meant for train 3 at time step 278. Since train 3 has only waited 2 time steps and has a speed of 4 it discards this action. You could say the global time is fast forwarded, while the local time of the train, the one keeping track of speed, is not. You can see this effect in action in timestep 278, where train 3 has now waited for 4 time steps, and therefore was able to move. To make it perfectly clear: From time step 274 to 278, the global time passed 8 units and the local time of train 3 only passed 4 units. It is annoying to compensate for in the solver directly, as it depends on all trains at a given time step. To solve this problem, we create an imaginary train with ID -1. This train has no position, no speed, no start, and no end, since flatland does not create this train. We do, however, give this ghost train an action for every time step in which another train exists, effectively getting rid of the time relativity issues. In theory, it would also be

sufficient to only add actions when no other train does a move. Since the train does not exist anyway, the extra actions do not cause problems.

```
action(train(-1), wait, T..T'):-thomas(_, _, T, _), thomas(_,
    _, T', _).
```

Listing 1.5: Adding extra actions for imaginary train.

agent	timestep	position	direction	status	given command
3	274	(30, 36)	n	moving	move forward
5	275	(33, 21)	w	moving	move forward
-1	276	None	None	moving	wait
-1	277	None	None	moving	wait
3	278	(30, 37)	e	moving	move forward
5	279	(33, 20)	w	moving	move forward

Table 3: Example of the solution. Exert from the new paths.csv

There was another approach we tried for resolving this issue. We tried filling up the time steps in between movements with 'wait' actions. While this works well for one train as soon as there are more than one train we encountered another problem. Flatland now uses the fill up action of one train correctly such that in every time step is an action. The other trains actions however are not needed for this purpose and can not be executed either since the train is not allowed to perform an action due to its speed. Therefore Flatland pushes them back to the next point where the train can move. This leads to the squaring of a trains speed. For example a train with speed 4 now moves every 16th time step.

4 Comparison between different implementations

Because Flatland allows 'wait' we can not solve every instance with the implementation outlined up until now. We implemented 'wait' in a naive way to show that our speed implementation is compatible with waits in theory. This implementation is not at all optimized leading to drastic increase in computing time. We will briefly show how we implemented waits and compare the computing times.

4.1 Without wait and without minimize

The environment in which we tested our implementation had 10 trains and 4 stations. You can see a possible solution in the animation below. The raw implementation was really effective for solving this with a CPU-Time of 3.016 seconds. It is important to note that like all other versions of our implementation the grounding is becoming quite expensive with increasing environment size.

This version was able to find a model for an environment with 14 trains and 4 stations with a CPU-Time of 30.359 seconds and an overall time of 47.664 seconds of which the solving only took 5.46 seconds.

4.2 With wait and without minimize

In an effort to be able to solve more Flatland instances, we added 'wait'. For that we allowed the thomases to either choose a connection or to stand still. This change is done in the thomas generation:

```
{thomas(ID, (X+X',Y+Y'), T+V, (X', Y')) : connection((X, Y),
     DIR, (X', Y')); thomas(ID, (X, Y), T+V, DIR)} = 1 :-
    thomas(ID, (X, Y), T, DIR), end(ID, POS, DEADLINE), T+V<
    DEADLINE, (X, Y) != POS, speed(ID, V).
```

Listing 1.6: rule for solving with wait

This version solved the 10, 4 environment with already drastically worse performance: CPU-Time 68.406s | Time: 111.599s | Solving: 4.15s. It is interesting though that, because we allowed 'wait', it found a better solution. The total number of thomas predicates was only 473 compared to 484 in the version without waits. This is quite surprising, as there is no incentive for the thomases to arrive as early as possible.

4.3 Wait and minimize

Even though the above version gave such a good result, we wanted to try to prevent trains from waiting without a purpose. We therefore also tried to implement a naive minimization targeted at minimizing the total number of thomas predicates.

```
#minimize{ 1, ID, T : thomas(ID, _, T, _) }.
```

Listing 1.7: rule for solving with wait

This increased the CPU-time to 193.188 seconds and only gave a model with 473 thomas predicates. So, minimization did not lead to a strictly better result compared to with wait and without minimize. Of course this can vary from environment to environment.

4.4 With minimize and without wait

We also tried to determine whether minimization yields better results with no 'wait' allowed. With a CPU-Time of 4.016 seconds it found a model without waits with 474 thomas predicates which is a noticeably better solution than the 484 without minimization.

4.5 Conclusion

The general conclusions from our tests were that the naive implementations of wait and minimization are not good enough for bigger environments. We tried larger environments and were simply not able to solve them in a reasonable amount of time. Also, it does not seem like minimization is necessary for solving with waits, contrary to what we expected. However, minimization seemed to work well for the version without waits both in terms of effiency and results. The following table contains all the measurements for the 10, 4 environment:

	no wait, no min	wait, no min	wait, min	no wait, min
CPU-TIME	3.016s	68.406s	193.188s	4.016s
Time	7.404s	111.599s	274.034s	15.526s
Solving	2.33s	4.15s	167.09s	11.82s
# thomas predicates	484	473	473	474

Table 4: result for 10, 4 environment

5 Future Work

These results clearly show that our speed implementation does what we want it to do, which is to implement speed but nothing more. We believe that this way of implementing speed is easily compatible with other approaches. These other approaches would need to figure out how to use this speed approach together with 'wait' while keeping grounding affordable for bigger environments. Another continuation of this work would be to combine our speed approach with malfunctions and deal with problems caused by that interaction. Finally, we thought about generating our actions in a Python script rather than in the solver itself. It does seem more neat and would certainly improve the solving time, but since the solving time is not a problem compared to the grounding, this is not a high priority.

6 Summary

This report focuses on improving ASP-based scheduling solutions by implementing variable train speeds, an aspect that was previously not implemented. The train speed is defined as the time required for a train to move between adjacent cellsfor example, a train with speed 4 takes four time steps to move one cell. We developed an approach that integrates speed into ASP-based scheduling, ensuring that train movement actions are generated with accurate timing and coordination.

To achieve this, the report details several key components of their implementation. The hierarchical tree like structure of the track types is mention.

Connection mapping defines the possible movements for trains on different track types. Collision handling ensures that no two trains occupy the same cell simultaneously or swap places in the same time step. Action generation determines the correct train movementswhether a train should move forward, turn left, turn right, or waitbased on its current state and track layout. Additionally, the study introduces the concept of a ghost train, a virtual train that exists purely to solve timing inconsistencies in Flatland. This approach compensates for the systems handling of idle time, ensuring that slower trains are not incorrectly skipped in the scheduling process.

The report also evaluates the performance of different scheduling implementations. Initial approaches that did not allow waiting times resulted in faster but sometimes inefficient schedules. Allowing wait actions improved train coordination but significantly increased computational costs. Attempts to minimize unnecessary waiting through optimization strategies proved computationally expensive and did not always yield better solutions. We conclude that while our approach successfully incorporates variable speeds, further work is required to optimize scheduling with wait times while maintaining reasonable computational efficiency.

Future research should explore how to effectively integrate train malfunctions into scheduling, as well as consider alternative methods such as Python-based action generation to improve performance. Overall, our work contributes to the development of more scalable and intelligent railway scheduling systems by leveraging ASP to manage train coordination in complex railway networks.